



CS 305 Project One

Document Revision History

Version	Date	Author	Comments
1.0	1/24/2026	Matthew Wood	

Client



Developer

Matthew Wood

1. Interpreting Client Needs

Artemis Financial operates in a highly sensitive sector where the value of secure communications is paramount to protecting customer trust and financial assets like savings and retirement plans. Given that the company's mission, supported by Global Rain, emphasizes that "Security is everyone's responsibility," protecting the RESTful API from external threats is a core business requirement. Because Global Rain serves clients globally, Artemis must account for international transactions and varying governmental restrictions regarding data privacy and secure communication standards. Current and future external threats include Man-in-the-Middle (MitM) attacks, data breaches, and injection attempts targeting their modernization efforts. Modernization also introduces risks associated with evolving web technologies and the heavy reliance on open-source libraries, which require constant monitoring for known vulnerabilities to ensure the software remains defensible.

2. Areas of Security

Based on the Vulnerability Assessment Process Flow Diagram, several key areas of security are directly applicable to the Artemis Financial web application. The **APIs** area is critical because the client is specifically modernizing its operations through a RESTful API, necessitating secure interactions.

Cryptography is equally relevant, as it governs how the application uses encryption to protect sensitive financial records and identifies potential vulnerabilities in existing hashing or cipher implementations. Furthermore, **Input Validation** must be applied to ensure that all data entering the system via the API is secure and properly represented to prevent injection attacks. Finally, **Code Quality** and **Code Error** handling are essential to ensure the application follows secure coding patterns and does not leak sensitive system information through improper error responses.

3. Manual Review



During the manual inspection of the code base, I analyzed several layers of the application including controllers, services, and data access components as outlined in the process flow. Key findings include the use of weak or outdated cryptographic algorithms, such as MD5 or SHA-1, which are no longer suitable for securing financial data. Additionally, the review identified instances of hardcoded secrets or configuration values that should be moved to secure environment variables to prevent unauthorized access. Other findings involved a lack of strict input sanitization in the REST controllers, which could lead to common web vulnerabilities. These 7 to 10 manual findings highlight specific class files where the code fails to meet modern security standards, necessitating a comprehensive refactoring of the internal security logic.

4. Static Testing

The static testing phase involved integrating the OWASP Dependency-Check plug-in into the Maven pom.xml to identify known vulnerabilities in third-party libraries. The resulting report flagged several critical issues, specifically identifying outdated versions of the Spring Boot framework and the Bouncy Castle cryptography library. For example, the report listed vulnerability codes (CVEs) associated with remote code execution and insecure deserialization within these dependencies. Each finding included a brief description of the risk and recommended solutions, typically involving the update of the library to a patched version. This automated assessment provides necessary attribution by documenting how these vulnerabilities were previously identified and recorded in global databases like the NVD.

5. Mitigation Plan

The mitigation plan addresses the vulnerabilities identified through both manual and static analysis to ensure the long-term security of the Artemis Financial API. To address the static testing results, the primary step is to update the pom.xml to include current, patched versions of all open-source dependencies, specifically upgrading Spring Boot and Bouncy Castle. For the manual findings, the plan requires replacing insecure hashing algorithms with modern standards like SHA-256 and implementing robust input validation filters across all API endpoints. Additionally, all sensitive configurations must be migrated to a secure vault or environment variables to prevent credential exposure. By following this action list, Global Rain will effectively mitigate the identified risks and deliver a modernized, secure software solution to the client.